Just grepping Around

The **grep** command is used to select lines that match a specified pattern from a stream of data. **grep** is one of the most commonly used filter utilities and can be used in some very creative and interesting ways. The **grep** command is one of the few that can correctly be called a filter because it does filter out all the lines of the data stream that you do not want; it leaves only the lines that you do want in the remaining data stream.

According to Klaatu, my reviewer for Volume 3 of this course, "One of the classic Unix commands, developed way back in 1974 by Ken Thompson, is the Global Regular Expression Print (grep) command. It's so ubiquitous in computing that it's frequently used as a verb ('grepping through a file') and, depending on how geeky your audience, it fits nicely into real-world scenarios, too. (For example, 'I'll have to grep my memory banks to recall that information.') In short, grep is a way to search through a file for a specific pattern of characters. If that sounds like the modern Find function available in any word processor or text editor, then you've already experienced grep's effects on the computing industry."[14]

EXPERIMENT 9-16: INTRODUCING GREP

We need to create a file with some random data in it. We can use a tool that generates random passwords, but we first need to install it as root:

```
dnf -y install pwgen
```

Now as the student user, let's generate some random data and create a file with it. If the PWD is not /test, make it so. The following command creates a stream of 5000 lines of random data that are each 75 characters long and stores them in the random.txt file:

```
pwgen 75 5000 > random.txt
```

Considering that there are so many passwords, it is very likely that some character strings in them are the same. Use the grep command to locate some short, randomly selected strings from the last ten passwords on the screen. I saw the words “see” and “loop” in one of those ten passwords, so my command looked like this:

```
grep see random.txt
```

You can try that, but you should also pick some strings of your own to check. Short strings of two to four characters work best.

Use the grep filter to locate all of the lines in the output from dmesg with CPU in them:

```
dmesg | grep cpu
```

List all of the directories in your home directory with the command

```
ls -la | grep ^d
```

This works because each directory has a “d” as the first character in a long listing. The caret (^) is used by grep and other tools to anchor the text being searched to the beginning of the line.

To list all of the files that are not directories, reverse the meaning of the previous grep command with the -v option:

```
ls -la | grep -v ^d
```

These are just a few examples of using sets. Continue to experiment with them to enhance your understanding.

Sets provide a powerful extension to pattern matching that gives us even more flexibility in searching for files. It is important to remember, however, that the primary use of these tools is not merely to "find" these files so we can look at their names. It is to locate files that match a pattern so that we can perform some operation on them, such as deleting, moving, adding text to them, searching their contents for specific character strings, and more.

Meta-characters

Meta-characters are ones that have special meaning to the shell. The bash shell has defined a number of these meta-characters, many of which we have already encountered in our explorations:

- $ Shell variable
- ~ Home directory variable
- & Run command in background
- ; Command termination/separation
- >, >>, < I/O redirection
- | Command pipe
- ‘, ”, \ Meta quotes
- $() Command substitution – preferred POSIX standard method
- `…` Command substitution
- (), {} Command grouping
- &&, || Shell control operators; conditional command execution

As we progress further through this course, we will explore the meta-characters we already know in more detail, and we will learn about the few we do not already know.

Using grep

Using file globbing patterns can be very powerful, as we have seen. We have been able to perform many tasks on large numbers of files very efficiently. As its name implies, however, file globbing is intended for use on file names, so it does not work on the content of those files. It is also somewhat limited in its capabilities.

There is a tool, `grep`, that can be used to extract and print to STDOUT all of the lines from a data stream based on matching patterns. Those patterns can range from simple text patterns to very complex regular expressions (regex). Written by Ken Thompson[3] and first released in 1974, the `grep` utility is provided by the GNU Project[4] and is installed by default on every version of Unix and Linux distribution I have ever used.

In terms of globbing characters, which grep does not understand, the default search pattern for the `grep` command is *PATTERN*. There is an implicit wildcard match before and after the search pattern. Thus, you can assume that any pattern you specify will be found no matter where it exists in the lines being scanned. It could be at the beginning, anywhere in the middle, or at the end. Thus, it is not necessary to explicitly state that there are characters in the string before and/or after the string for which we are searching.

EXPERIMENT 15-10: USING GREP

Perform this experiment as root. Although non-privileged users have access to some of the data we will be searching, only root has access to all of it.

One of the most common tasks I do that requires the use of the grep utility is scanning through log files to find information pertaining to specific things. For example, I may need to determine information about how the operating system sees the network interface cards (NICs) starting with their BIOS names,[5] ethX. Information about the NICs installed in the host can be found using the `dmesg` command as well as in the messages log files in /var/log.

We'll start by looking at the output from `dmesg`. First, just pipe the output through `less` and use the search facility built into `less`:

```
[root@studentvm1 ~]# dmesg | less
```

You can page through the screens generated by `less` and use the Mark I Eyeball[6] to locate the "eth" string, or you can use the search. Initiate the search facility by typing the slash (/) character and then the string for which you are searching: /eth. The search will highlight the string, and you can use the "n" key to find the next instance of the string and the "b" key to search backward for the previous instance.

Searching through pages of data, even with a good search facility, is easier than eyeballing it, but not as easy as using `grep`. The -i option tells grep to ignore case and display the "eth" string regardless of the case of its letters. It will find the strings eth, ETH, Eth, eTh, and so on, which are all different in Linux:

```
[root@studentvm1 ~]# dmesg | grep -i eth

[    1.861192] e1000 0000:00:03.0 eth0: (PCI:33MHz:32-bit) 08:00:27:a9:e6:b4

[    1.861199] e1000 0000:00:03.0 eth0: Intel(R) PRO/1000 Network Connection

[    2.202563] e1000 0000:00:08.0 eth1: (PCI:33MHz:32-bit) 08:00:27:50:58:d4

[    2.202568] e1000 0000:00:08.0 eth1: Intel(R) PRO/1000 Network Connection

[    2.205334] e1000 0000:00:03.0 enp0s3: renamed from eth0

[    2.209591] e1000 0000:00:08.0 enp0s8: renamed from eth1

[root@studentvm1 ~]#
```

These results show data about the BIOS names, the PCI bus on which they are located, the MAC addresses, and the new names that Linux has given them. Now look for instances of the string that begins the new NIC names, “enp”. Did you find any?

Note The numbers enclosed in square braces, [2.205334], are timestamps that indicate the log entry was made that number of seconds after the kernel took over control of the computer.

In this first example of usage, `grep` takes the incoming data stream using STDIN and then sends the output to STDOUT. The grep utility can also use a file as the source of the data stream. We can see that in this next example in which we grep through the messages log files for information about our NICs:

```
[root@studentvm1 ~]$ cd /var/log ; grep -i eth messages*

<snip>

messages-20181111:Nov  6 09:27:36 studentvm1 dbus-daemon[830]: [system] Rejected send mess

messages-20181111:Nov  6 09:27:36 studentvm1 pulseaudio[1738]: E: [pulseaudio] bluez5-util

messages-20181118:Nov 16 07:41:00 studentvm1 kernel: e1000 0000:00:03.0 eth0: (PCI:33MHz:3

messages-20181118:Nov 16 07:41:00 studentvm1 kernel: e1000 0000:00:03.0 eth0: Intel(R) PRO

messages-20181118:Nov 16 07:41:00 studentvm1 kernel: e1000 0000:00:08.0 eth1: (PCI:33MHz:3

messages-20181118:Nov 16 07:41:00 studentvm1 kernel: e1000 0000:00:08.0 eth1: Intel(R) PRO

<SNIP>
```

The first part of each line in our output data stream is the name of the file in which the matched lines were found. If you do a little exploration of the current messages file, which is named just that with no appended date, you may or may not find any lines matching our search pattern. I did not with my VM, so using the file glob to create the pattern "messages*" searches all of the files starting with messages. This file glob matching is performed by the shell and not by the `grep` tool.

You will notice also that, on this first try, we found more than we wanted. Some lines have the "eth" string in them that was found as part of the word "method." So let's be a little more explicit and use a set as part of our search pattern:

```
[root@studentvm1 log]# grep -i eth[0-9] messages*
```

This is better, but we also want the lines that pertain to our NICs after they were renamed. So we now know the names that our old NIC names were changed to, so we can also search for those. Fortunately for us, grep provides some interesting options such as using -e to specify multiple search expressions. Each search expression must be specified using a separate instance of the -e option:

Tip Each expression is additive. That is, the eth[0-9] expression finds all messages that contain that phrase, and the enp0 expression finds all messages that contain that one. So lines containing either one or the other or both expressions are displayed. Therefore, this next command will produce a long data stream.

```
[root@studentvm1 log]# grep -i -e eth[0-9] -e enp0 messages*
```

That does work, but there is also an extension that allows us to search using extended regular expressions.[7] The **grep** patterns we have been using so far are basic regular expressions (BRE). To get more complex, we can use extended regular expressions (ERE). To do this we can use egrep:

```
[root@studentvm1 log]# egrep "eth[0-9] | enp0" messages*
```

You may wish to use the wc (word count) command to verify that both of the last two commands produce the same number of lines for their results.

Note that the extended regular expression is enclosed in double quotes.

Now make /etc the PWD. Sometimes I have previously needed to list all of the configuration files in the /etc directory. These files typically end with a .conf or .cnf extension or with rc. To do this we need an anchor to specify that the search string is at the end of the string being searched. We use the dollar sign ($) for that. The syntax of the search string in the following command finds all the configuration files with the listed endings. The -R option for the **ll** or **ls** command causes the command to recurse into all of the subdirectories:

```
[root@studentvm1 etc]# ls -aR | grep -E "conf$|cnf$|rc$"
```

Use word count to display the number of files selected and then use the equivalent egrep command and see that it selects the same number of files.

We can also use the caret (^) to anchor the beginning of the string. Suppose that we want to locate all files in /etc that begin with kde because they are used in the configuration of the KDE desktop:

```
[root@studentvm1 etc]# ls -R | grep -E "^kde"

kde
```

```
kde4rc

kderc

kde.csh

kde.sh

kdebugrc
```

One of the advanced features of **grep** is the ability to read the search patterns from a file containing one or more patterns. This is very useful if the same complex searches must be performed on a regular basis.

The **grep** tool is powerful and complex. The man page offers a good amount of information, and the GNU Project provides a free 36-page manual[8] to assist with learning and using **grep**. That document is available in HTML so it can be read online with a web browser, as ASCII text, as an info document, as a downloadable PDF file, and more.

Finding Files

The `ls` command and its aliases such as `ll` are designed to list all of the files in a directory. Special pattern characters and the grep command can be used to narrow down the list of files sent to STDOUT. But there is still something missing. There is a bit of a problem with the command `ls -R | grep -E "^kde"` that we used in Experiment 15-10. Some of the files it found were in subdirectories of /etc/, but the ls command does not display the names of the subdirectories in which those files are stored.

Fortunately the `find` command is designed explicitly to search for files in a directory tree using patterns and to either list the files and their directories or to perform some operation on them. The `find` command can also use attributes such as the date and time a file was created or accessed, files that were created or modified before or after a date and time, its size, permissions, user ID, group ID, and much more. These attributes can be combined to become very explicit, such as all files that are larger than 12M in size, that were created more than 5 years ago, that